Intro to Data Science - Lab 5

Copyright 2025, Jeffrey Stanton and Jeffrey Saltz Please do not post online.

Week 5 - Obtaining and Using Data from a JSON API

```
# Enter your name here: Sofia Quintero
```

Please include nice comments.

Instructions:

Run the necessary code on your own instance of R-Studio.

Attribution statement: (choose only one and delete the rest)

```
# 1. I did this lab assignment by myself, with help from the book and the professor.
```

JSON (JavaScript Object Notation) provides a transparent, user friendly data exchange format that many organizations use to make their data available over the web. JSON is human readable, but is also highly structured, and its support for nested data structures makes it highly flexible. Today we will use JSON to obtain data from the **NYC CitiBike project**. The CitiBike project provides an application programming interface (API) that members of the public can access to get up-todate information on the status of more than 800 bike stations. You may need to install the **RCurl** and **jsonlite** packages to get the code to work.

```
library('RCurl')
library('jsonlite')
station_link <- 'https://gbfs.citibikenyc.com/gbfs/en/station_status.json'
apiOutput <- getURL(station_link)
apiData <- fromJSON(apiOutput)
stationStatus <- apiData$data$stations
cols <- c('num_bikes_disabled','num_docks_disabled', 'station_id',
 'num_ebikes_available', 'num_bikes_available', 'num_docks_available')
stationStatus = stationStatus[,cols]
```

1. Explain what you see if you type in the **station_link** URL into a browser (in a comment, write what you see)

```
#When the station_link URL is put into a browser it returns a wall of text
#contating what appears to be code for variable assignments
```

2. Paste the code from above here and provide a comment explaining each line of code.

```r
station_link <- 'https://gbfs.citibikenyc.com/gbfs/en/station_status.json' #variable assignment
apiOutput <- getURL(station_link) #getURL() accessed and downloads a given URL
apiData <- fromJSON(apiOutput) #fromJSON() converts code to R from JSON
stationStatus <- apiData$data$stations #creates dataframe from station data
cols <- c('num_bikes_disabled','num_docks_disabled', 'station_id',
 'num_ebikes_available', 'num_bikes_available', 'num_docks_available') #established columns
stationStatus = stationStatus[,cols] #reassigns dataframe with specific columns
```

3. Use **str( )** to find out the structure of **apiOutput** and **apiData**. Report (via a comment) what you found and explain the difference between these two objects.

```r
str(apiOutput)
```

```
##  chr "{\"data\": {\"stations\": [{\"num_bikes_available\": 50, \"num_ebikes_available\": 22, \"is_ins
```

```r
#Returns data contents in JSON format with content for one row
```

```r
str(apiData)
```

```
## List of 4
##  $ data        :List of 1
##   ..$ stations:'data.frame': 2231 obs. of  14 variables:
##   .. ..$ num_bikes_available      : int [1:2231] 50 6 11 3 12 32 14 15 6 21 ...
##   .. ..$ num_ebikes_available     : int [1:2231] 22 3 0 1 0 9 12 2 0 1 ...
##   .. ..$ is_installed             : int [1:2231] 1 1 1 1 1 1 1 1 1 1 ...
##   .. ..$ is_renting               : int [1:2231] 1 1 1 1 1 1 1 1 1 1 ...
##   .. ..$ num_scooters_unavailable : int [1:2231] 0 0 0 0 0 0 0 0 0 0 ...
##   .. ..$ last_reported            : int [1:2231] 1739897860 1739897860 1739897863 1739897863 1739897
##   .. ..$ num_scooters_available   : int [1:2231] 0 0 0 0 0 0 0 0 0 0 ...
##   .. ..$ num_docks_available      : int [1:2231] 5 28 6 18 11 1 17 3 9 3 ...
##   .. ..$ station_id               : chr [1:2231] "66dc8db1-0aca-11e7-82f6-3863bb44ef7c" "66dd3d7f-0a
##   .. ..$ eightd_has_available_keys: logi [1:2231] FALSE FALSE FALSE FALSE FALSE FALSE ...
##   .. ..$ is_returning             : int [1:2231] 1 1 1 1 1 1 1 1 1 1 ...
##   .. ..$ num_bikes_disabled       : int [1:2231] 2 2 2 3 0 3 1 1 3 0 ...
##   .. ..$ num_docks_disabled       : int [1:2231] 0 0 0 0 0 0 0 0 0 0 ...
##   .. ..$ legacy_id                : chr [1:2231] "2000" "3131" "4147" "1846085439406916338" ...
##  $ last_updated: int 1739897949
##  $ ttl         : int 60
##  $ version     : chr "1.1"
```

```r
#Returns columns in data including its contents of all columns and rows
#Because this is in R format, the code is able to read this information
```

4. The **apiOutput** object can also be examined with a custom function from the **jsonlite** package called **prettify( )**. Run this command and explain what you found (in a comment).

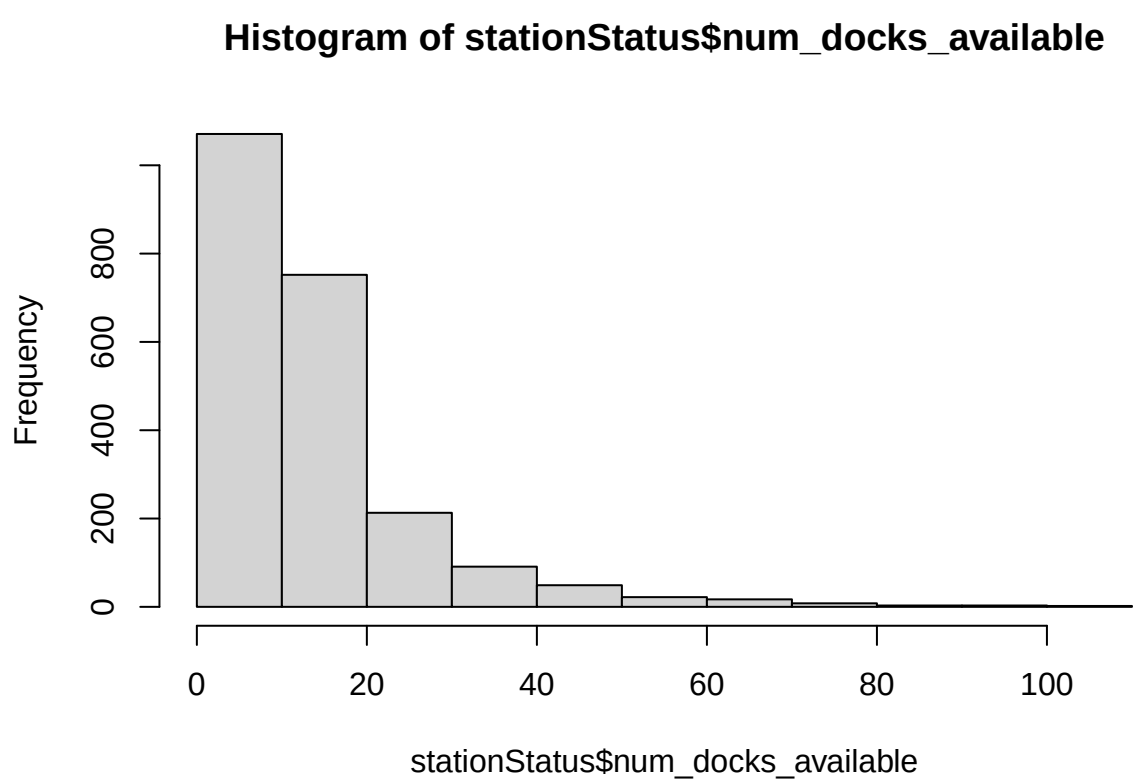
```r
#prettify(apiOutput)
#The JSON code is now sectioned off by what each row in a dataframe would display
```

5. Explain **stationStatus** (what type of object, what information is available).

```r
#stationStatus inlcudes int, logi, and chr objects
#The data provides information for the station such as how many bikes, ebikes,
#and scooters are available, how many are installed and
#renting, how many bikes and docks are available.
```

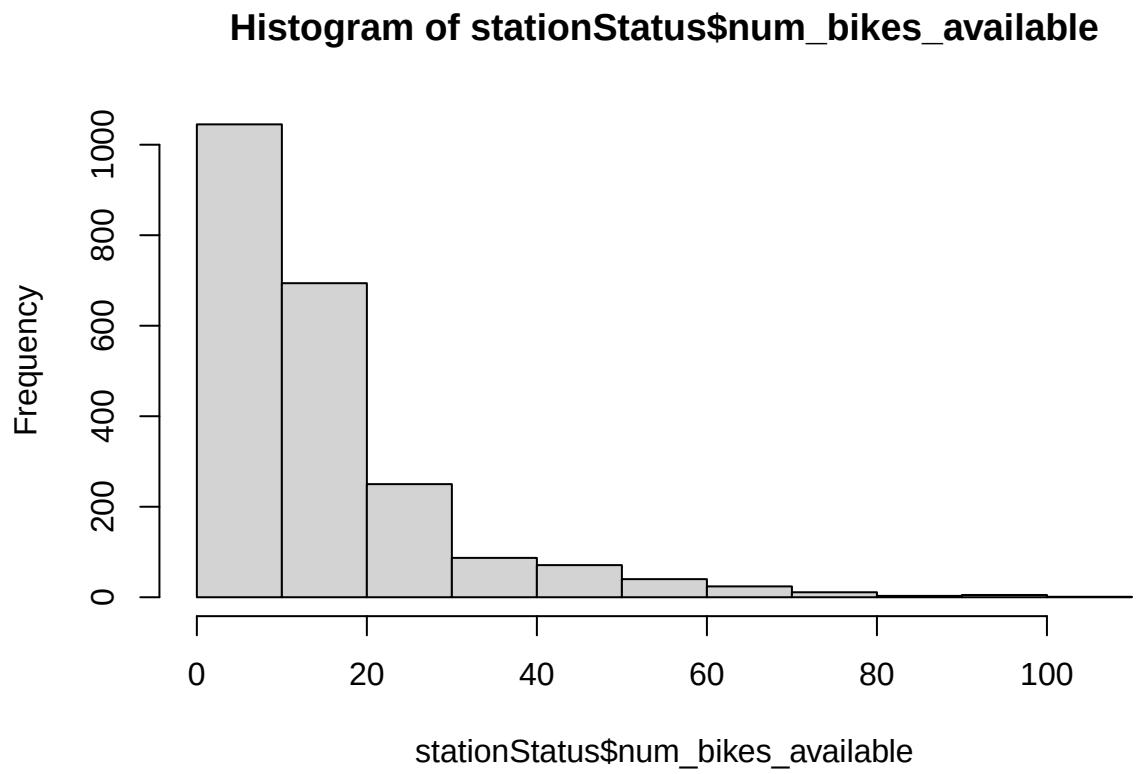
6. Generate a histogram of the number of docks available

```r
hist(stationStatus$num_docks_available)
```



7. Generate a histogram of the number of bikes available

```r
hist(stationStatus$num_bikes_available)
```



8. How many stations have at least one ebike?

```
nrow(subset(stationStatus, num_ebikes_available > 0))
```

```
## [1] 1509
```

```
#There are 1519 stations with at leasts one ebike available
```

9. Explore stations with at least one ebike by create a new dataframe, that only has stations with at least one eBike.

```
ebikeAvail <- subset(stationStatus, num_ebikes_available > 0)
```

10. Calculate the mean of ** num_docks_available ** for this new dataframe.

```
mean(ebikeAvail$num_docks_available)
```

```
## [1] 12.2061
```

11. Calculate the mean of ** num_docks_available ** for for the full ** stationStatus ** dataframe. In a comment, explain how different are the two means?

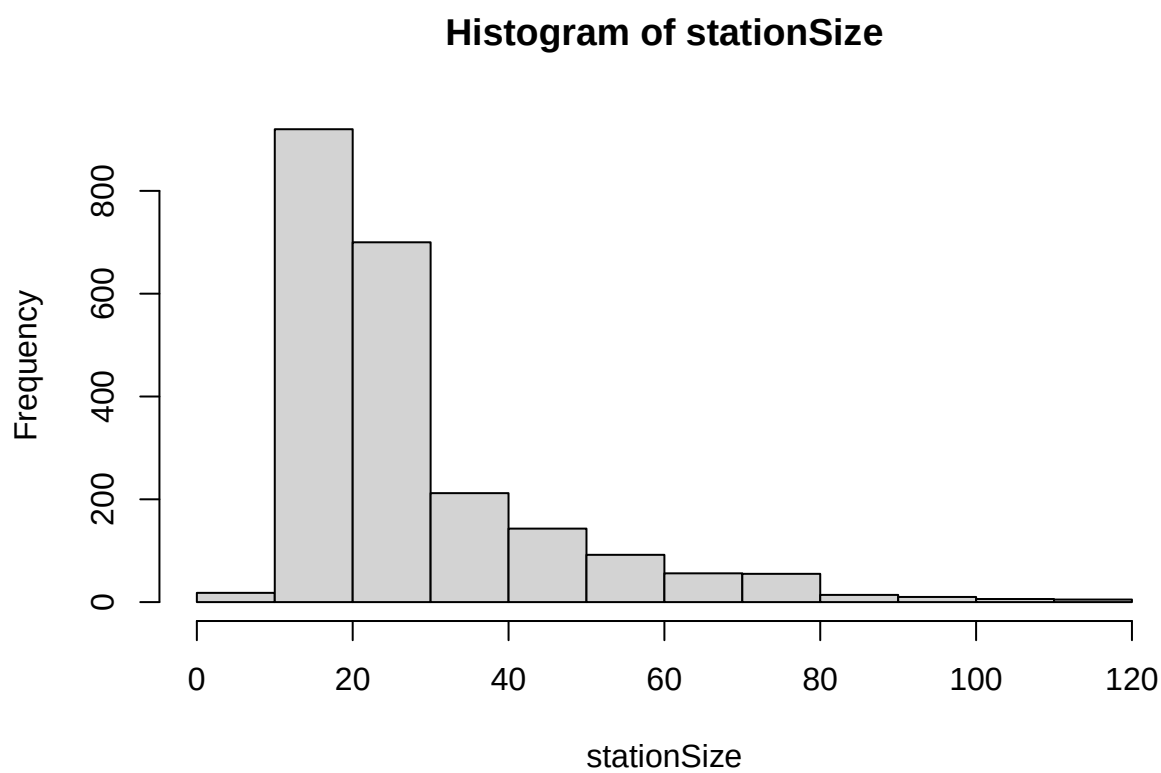
```r
mean(stationStatus$num_docks_available)
```

```
## [1] 13.54505
```

```r
#The mean of docks available for station status is greater than those with ebikes available
#This may be due to less empty docks being present for stations with at least one ebike
#as the ebikes are occupying them.
```

12. Create a new attribute, called ** stationSize **, which is the total number of slots available for a bike (that might, or might not, have a bike in it now). Run a histogram on this variable and review the distribution.

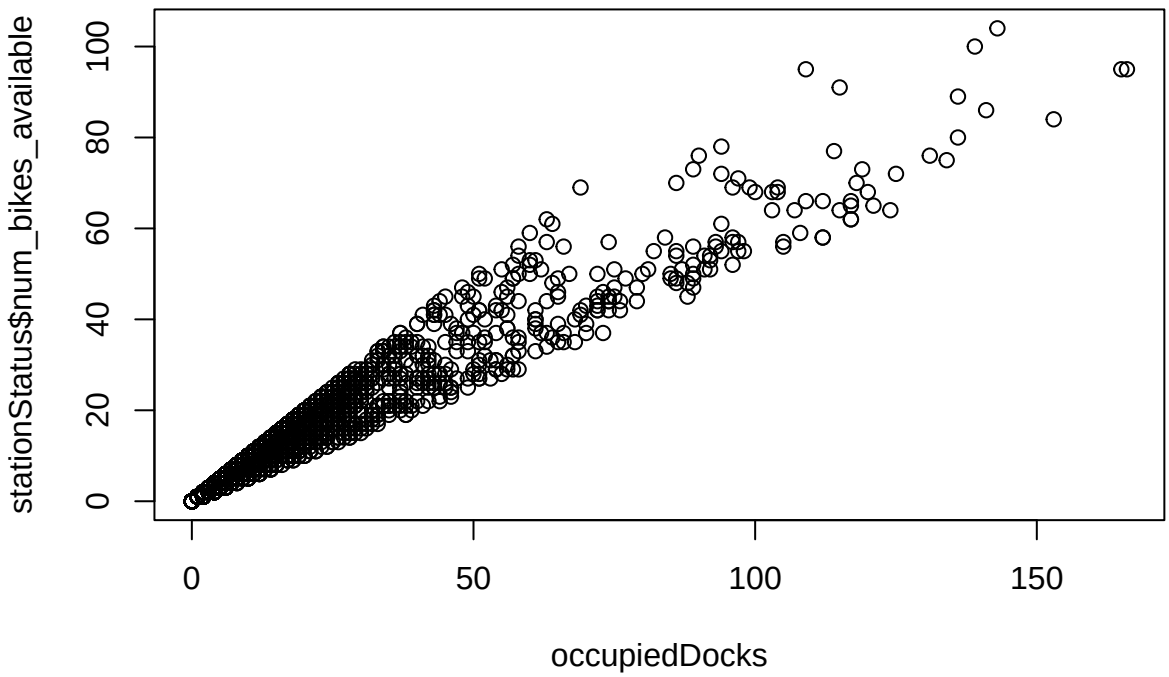
```r
stationSize <- (stationStatus$num_bikes_available + stationStatus$num_docks_available)
hist(stationSize)
```




```r
#The majority of stations tend to hold about 20 bikes
```

13. Use the **plot( )** command to produce an X-Y scatter plot with the number of occupied docks on the X-axis and the number of available bikes on the Y-axis. Explain the results plot.

```r
occupiedDocks <- (stationStatus$num_ebikes_available + stationStatus$num_bikes_available)
plot(occupiedDocks, stationStatus$num_bikes_available)
```




```
#The more docks that are occupied, the more likely a bike is available
#This is why the plot shows positive correlation
```